

深度研究框架对比 与 AURA 新框架设计

四大开源实现深度剖析 · 一般流程提取 · 新框架实现计划

本报告系统对比 langchain-ai/open_deep_research、
assafelovic/gpt-researcher、qx-labs/agents-deep-research、
dzhng/deep-research 四大开源深度研究实现的工作流程，
结合 MiroFlow 论文的关键启示，提取深度研究的一般流程框架，
并提出新框架 AURA (Adaptive Unified Research Architecture)
的设计方案与实现路线图。

目录

第一章 引言：深度研究的兴起与背景4

1.1 什么是"深度研究"4

1.2 为什么需要研究这些开源实现4

1.3 报告结构4

第二章 四大开源实现深度剖析6

2.1 langchain-ai/open_deep_research6

2.2 assafelovic/gpt-researcher8

2.3 qx-labs/agents-deep-research10

2.4 dzhng/deep-research12

第三章 流程异同对比分析13

3.1 共性流程提取13

3.2 差异化设计对比13

3.3 综合对比矩阵15

3.4 关键发现15

第四章 深度研究的一般流程框架16

4.1 一般流程的六阶段模型16

4.2 一般流程的可视化17

4.3 关键设计决策18

第五章 MiroFlow 论文的关键启示19

5.1 MiroFlow 概述19

5.2 三层次化架构19

5.3 三项关键技术创新19

5.4 对新框架设计的启示20

第六章 AURA 新框架设计与实现计划21

6.1 设计理念与目标21

6.2 架构总览21

6.3 核心模块设计23

6.4 关键创新点详解25

6.5 实现路线图25

6.6 技术栈建议26

第七章 预期优势与评估方案28

7.1 AURA 相对于现有方案的预期优势28

7.2 评估方案29

7.3 预期性能目标29

7.4 风险与缓解30

7.5 最终结论30

第一章 引言：深度研究的兴起与背景

1.1 什么是"深度研究"

深度研究（Deep Research）是 2024 年末至 2025 年间大语言模型应用领域最具突破性的方向之一。它指的是：给定一个复杂的研究问题，AI 系统通过自主规划、多轮网络搜索、信息提取与交叉验证、迭代式推理与综合，最终生成一份结构化、有引用、有深度的研究报告的完整过程。与传统的单次检索增强生成（RAG）不同，深度研究强调"多步迭代"和"自主决策"——系统需要像人类研究员一样，根据已有发现动态调整搜索策略，识别知识盲区，并在信息充分后停止探索。

这一概念的爆发始于 OpenAI 在 2024 年 12 月推出的 Deep Research 功能。该功能基于 o3 模型的强化推理能力，能够自主浏览数十个网页、综合数百个信息源，在 5-30 分钟内完成过去人类研究员需要数小时才能完成的工作。随后 Google Gemini Deep Research、Perplexity Pro Deep Research 等商业产品相继推出，开源社区也迅速跟进，涌现出多个高质量的实现方案。

1.2 为什么需要研究这些开源实现

商业产品的闭源特性使得研究者和开发者难以理解其内部机制，更无法进行定制化改进。开源实现的出现填补了这一空白，为社区提供了可复现、可改进的基础设施。然而，不同开源实现在架构设计、流程编排、智能体协作模式等方面存在显著差异，各有优劣。理解这些差异对于构建更优的深度研究系统至关重要。

本报告选取了四个具有代表性的开源实现进行深度剖析：langchain-ai/open_deep_research 代表了基于 LangGraph 的层次化多智能体方案；assafelovic/gpt-researcher 是最早的开源深度研究框架之一，拥有最成熟的工程化实践；qx-labs/agents-deep-research 基于 OpenAI Agents SDK，展示了知识盲区驱动的迭代研究模式；dzhng/deep-research 则以不到 500 行代码的极简实现，清晰展示了深度研究的核心递归逻辑。此外，我们还参考了 MiroFlow 论文（arXiv:2602.22808），该论文提出了一个在 GAIA、BrowseComp 等多个基准测试上达到 SOTA 性能的智能体框架，其设计理念对新框架的设计具有重要启示。

1.3 报告结构

本报告共分为七章。第二章对四个开源实现逐一进行深度剖析，包括架构设计、核心流程、关键特性与适用场景。第三章从多个维度对比四个实现的异同，提取共性模式与差异化设计。第四章基于对比分析，提炼出深度研究的一般流程框架。第五章解读 MiroFlow 论文的关键技术贡献。第六章提出新框架 AURA 的完整设计方案与实现路线图。第七章分析 AURA 的预期优势并给出评估方案。

核心目标：通过系统对比已有方案的优势与不足，设计一个在自适应能力、结果质量、执行鲁棒性和成本效率四个维度上全面超越现有方案的新框架 AURA，并给出可落地的实现计划。

第二章 四大开源实现深度剖析

2.1 langchain-ai/open_deep_research

2.1.1 项目概览

open_deep_research 是 LangChain 官方推出的深度研究框架，其核心定位是复现并超越 OpenAI Deep Research 的能力。该项目基于 LangGraph 构建，采用层次化的"主管-研究员"（Supervisor-Researcher）多智能体架构，支持多种搜索工具（Tavily、Serper 等）和 MCP（Model Context Protocol）工具服务器，可配置不同的 LLM 后端。项目强调可定制性和可扩展性，允许用户替换模型、搜索工具、提示词模板等各个组件。

2.1.2 核心架构与流程

该框架的执行流程由 LangGraph 状态图驱动，包含以下关键节点：

- 澄清交互（clarify_with_user）**：系统首先分析用户查询，如果研究范围不够明确，会生成澄清问题并通过 human-in-the-loop 机制等待用户回答。这是四个实现中唯一内置用户交互环节的方案。
- 研究简报生成（write_research_brief）**：将用户查询（及澄清回答）转化为结构化的研究简报，明确研究目标、范围和预期产出格式。
- 研究主管子图（research_supervisor）**：这是核心的层次化多智能体模块。主管智能体负责制定研究策略，通过 think_tool 进行战略反思，然后调用 ConductResearch 委派具体研究任务给多个并行的研究员智能体。
- 研究员子图（researcher subgraph）**：每个研究员智能体是一个独立的子图，包含 researcher 节点（执行搜索）、researcher_tools 节点（工具调用）和 compress_research 节点（压缩研究发现）。研究员可以使用搜索工具、MCP 工具和 think_tool。
- 研究压缩（compress_research）**：每个研究员完成搜索后，将原始发现压缩为简洁的摘要，以管理上下文窗口限制。这一机制使得系统可以在有限的上下文内处理大量信息。
- 最终报告生成（final_report_generation）**：主管收集所有研究员的压缩摘要后，综合生成最终的结构化研究报告。

2.1.3 关键特性

- **层次化多智能体**：主管-研究员的两层结构，支持并行研究（max_concurrent_research_units 可配置）。
- **战略反思**：通过 think_tool 让智能体在行动前进行显式推理和策略调整。
- **渐进式截断**：当上下文接近 token 限制时，自动截断较早的研究内容，保证系统不会因上下文溢出而崩溃。
- **重试机制**：关键节点使用 with_retry 包装，提高执行稳定性。
- **MCP 工具集成**：支持通过 MCP 协议接入外部工具服务器，极大扩展了系统能力边界。

- **人机交互**：内置澄清环节，适合需要精确理解用户意图的场景。

2.1.4 适用场景与局限

`open_deep_research` 适合需要高度定制化、多工具集成和精确意图理解的研究场景。其层次化架构使得系统可以处理复杂的多维度研究任务。然而，其架构复杂度较高，`LangGraph` 的学习曲线较陡，且层次化通信带来的 token 开销较大。此外，系统的性能高度依赖主管智能体的规划质量，如果主管生成的策略不佳，后续研究员的工作可能偏离方向。

2.2 assafelovic/gpt-researcher

2.2.1 项目概览

gpt-researcher 是最早的开源深度研究框架之一，由 Tavily AI 创始人 Assaf Elovic 创建。该项目灵感来源于 Princeton 大学 2023 年的论文"Plan-and-Solve Prompting"，采用"规划-执行"范式，支持标准研究和深度研究两种模式。项目拥有最成熟的工程化实践，包括多种检索器（Tavily、Serper、DuckDuckGo 等）、多种抓取器、详细的文档和活跃的社区。标准研究模式约 3 分钟、成本约 0.1 美元；深度研究模式约 5 分钟、成本约 0.4 美元（使用 o3-mini high）。

2.2.2 核心架构与流程

标准研究模式（Standard Research）

标准模式采用线性的"规划-执行"流程：

1. **智能体创建**：根据查询内容创建领域特定的智能体配置。
2. **规划**：规划器（Planner）生成一组研究问题，覆盖查询的各个维度。
3. **并行抓取**：对每个研究问题，爬虫智能体（Crawler Agent）搜索并抓取相关网页资源。
4. **摘要与引用**：对每个抓取的资源进行摘要，并跟踪信息来源。
5. **过滤与聚合**：过滤低质量内容，聚合所有摘要后的信息源。
6. **报告生成**：基于聚合的信息生成最终报告，包含内联引用。

深度研究模式（Deep Research）

深度模式引入了树状探索结构，通过 `breadth`（广度）和 `depth`（深度）两个参数控制探索规模：

- **breadth**（默认 4）：每个层级并行探索的研究路径数量。
- **depth**（默认 2）：递归探索的层级数。
- **concurrency**（默认 4）：最大并行操作数。

深度模式的核心是一个递归函数，在每一层生成多个查询，并行执行搜索和摘要，然后将发现作为下一层的上下文继续探索。系统通过 `async/await` 实现并发处理，并具备智能的上下文管理（跨分支共享发现）和进度跟踪机制。错误处理方面，失败的查询会被跳过而非终止整个流程。

2.2.3 关键特性

- **多检索器支持**：集成 Tavily、Serper、DuckDuckGo、Bing 等多种搜索引擎 API。
- **多抓取器支持**：支持 BeautifulSoup、Selenium、Playwright 等多种网页抓取方式。
- **成本优化**：规划阶段使用 gpt-4o-mini，综合阶段使用 gpt-4o，平衡成本与质量。
- **偏见降低**：通过聚合多个信息源来降低单一来源的偏见。
- **内联引用**：报告中直接标注信息来源，便于验证。
- **详细文档**：拥有最完善的文档和教程，社区活跃度高。

2.2.4 适用场景与局限

gpt-researcher 适合需要快速部署、成本敏感的场景。其标准模式在 3 分钟内即可完成研究，适合实时性要求较高的应用。深度模式通过树状探索提供了更好的覆盖度。然而，该框架的规划阶段是静态的——所有研究问题在开始时一次性生成，不会根据中间发现动态调整。此外，系统缺乏显式的反思和质量评估机制，研究深度完全由 breadth 和 depth 参数决定，而非由内容质量驱动。

2.3 qx-labs/agents-deep-research

2.3.1 项目概览

agents-deep-research 基于 OpenAI Agents SDK 构建，提供了两种研究模式：面向短报告（5 页以内）的 IterativeResearcher 和面向长报告（20 页以上）的 DeepResearcher。该项目的核心创新在于"知识盲区驱动"的迭代研究模式——系统显式地识别当前知识的不足之处，然后针对性地选择工具进行补充研究，形成"发现盲区-选择工具-执行研究-综合观察"的闭环。

2.3.2 核心架构与流程

IterativeResearcher（短报告模式）

该模式通过一个持续的智能体循环驱动研究：

- 知识盲区智能体（Knowledge Gap Agent）**：分析当前已收集的信息，识别研究中的知识盲区——即哪些关键问题尚未得到充分回答。
- 工具选择智能体（Tool Selector Agent）**：根据识别出的知识盲区，战略性地选择最合适的工具（Web Search、Crawler 或自定义工具）。
- 工具智能体（Tool Agents）**：并行执行选定的工具，获取新信息。
- 观察综合智能体（Observations Agent）**：综合新获取的信息，提供对研究策略的思考和建议。
- 循环**：回到步骤 1，继续识别新的知识盲区，直到满足终止条件（最大迭代数、最大时间或知识盲区已充分覆盖）。
- 写作智能体（Writer Agent）**：最终将所有研究发现综合为带引用的研究报告。

DeepResearcher（长报告模式）

对于需要长篇报告的场景，系统采用"规划-并行研究-校对"的三阶段模式：

- 规划智能体（Planner Agent）**：进行初步研究，生成报告大纲（outline），将长报告分解为多个章节。
- 并行 IterativeResearcher**：为每个章节并行启动一个独立的 IterativeResearcher 实例，各自深入研究对应主题。
- 校对智能体（Proofreader Agent）**：将所有章节的研究结果编译为一份连贯的长篇报告，确保各章节之间的逻辑一致性和内容衔接。

2.3.3 关键特性

- **多模型支持**：支持 OpenAI、Anthropic、Gemini、DeepSeek 等多种 LLM 后端。
- **三层模型分级**：reasoning 层（o3-mini）用于复杂推理，main 层（gpt-4o）用于主要任务，fast 层（gpt-4o-mini）用于轻量任务。
- **OpenAI Tracing**：内置执行追踪支持，便于调试和分析。

- **可配置终止条件**: max_iterations、max_time、output_length 等参数灵活控制。
- **知识盲区驱动**: 显式识别知识不足, 使研究方向更有针对性。

2.3.4 适用场景与局限

agents-deep-research 适合需要高质量、长篇报告的场景。其知识盲区驱动的迭代模式使得研究更有针对性, 避免了无目的的搜索。三层模型分级有效平衡了成本与质量。然而, 该系统明确表示不进行澄清提问 ("No clarifying questions"), 完全自动化执行, 这在某些需要精确理解用户意图的场景下可能不足。此外, 系统强依赖 OpenAI Agents SDK, 可移植性受限。

2.4 dzhng/deep-research

2.4.1 项目概览

dzhng/deep-research 是四个实现中最简洁的一个，核心代码不到 500 行 TypeScript。它由 Vercel AI SDK 示例演化而来，采用纯粹的递归树状探索结构。项目使用 Firecrawl 进行搜索和网页抓取，支持 OpenAI 兼容 API（包括 OpenRouter、Gemini 等）。其极简的设计使得核心逻辑清晰可读，非常适合作为理解深度研究原理的入门参考。

2.4.2 核心架构与流程

该框架的核心是一个递归函数 `deepResearch`，其执行流程如下：

- 需求澄清：**系统首先根据用户查询生成后续问题，以更好地理解研究需求。用户回答后，系统将原始查询和回答合并为完整的研究目标。
- 递归研究（`deepResearch`）：**核心递归函数接收研究目标、当前广度（`breadth`）、当前深度（`depth`）和已有发现（`learnings`）作为参数。
- 查询生成：**在每一层，系统根据当前研究目标生成多个 SERP 搜索查询（数量由 `breadth` 决定）。
- 搜索与提取：**对每个查询执行搜索，从结果中提取“学习要点”（`Learnings`）和“新方向”（`Directions`）。
- 递归探索：**如果 `depth > 0`，系统从提取的新方向中选择若干个，以当前学习要点为上下文，递归调用 `deepResearch` 继续探索。
- 发现累积：**所有层级的学习要点被累积收集，形成完整的知识库。
- 报告编译：**最终将所有学习要点编译为带引用的 Markdown 报告。

2.4.3 关键特性

- **极简实现：**核心逻辑不到 500 行，易于理解和扩展。
- **递归树结构：**`breadth` 控制每层查询数，`depth` 控制递归层数，形成自然的树状探索结构。
- **方向提取：**每轮搜索后显式提取新的研究方向，使探索具有自适应性。
- **Firecrawl 集成：**使用 Firecrawl 进行高效的搜索和内容提取。
- **OpenAI 兼容：**支持任何 OpenAI 兼容的 API 端点。

2.4.4 适用场景与局限

dzhng/deep-research 适合快速原型开发和学习理解深度研究原理。其极简设计使得开发者可以快速上手并进行定制化修改。递归树结构直观地展示了深度研究的核心思想。然而，该实现的功能较为基础：缺乏并行处理能力（搜索是串行的）、没有质量评估机制、没有错误恢复策略、上下文管理简单（所有学习要点直接累积，可能超出上下文窗口）。此外，系统强依赖 Firecrawl 服务，灵活性受限。

第三章 流程异同对比分析

3.1 共性流程提取

尽管四个实现在架构设计和编排方式上差异显著，但深入分析可以发现它们共享一组核心流程步骤。这些共性步骤构成了深度研究功能的基础骨架，任何完整的深度研究系统都需要涵盖这些环节。

3.1.1 六大共性环节

- 1. 查询理解与规划：**所有实现都以分析用户查询为起点，生成研究计划或子问题。open_deep_research 生成研究简报，gpt-researcher 生成研究问题列表，agents-deep-research 由 Planner 生成大纲，dzhng/deep-research 生成 SERP 查询。
- 2. 搜索与检索：**所有实现都通过网络搜索获取信息，使用 Tavily、Serper、Firecrawl 等搜索 API。这是深度研究与传统 RAG 的核心区别——信息来源是实时网络而非预建知识库。
- 3. 内容提取与处理：**搜索结果需要被抓取、解析和提取。所有实现都包含网页内容提取环节，将原始 HTML 转化为可处理的文本。
- 4. 信息综合与摘要：**所有实现对提取的信息进行摘要和综合，以管理上下文窗口限制并提炼关键信息。open_deep_research 的 compress_research、gpt-researcher 的资源摘要、agents-deep-research 的 Observations Agent、dzhng/deep-research 的 Learnings 提取都服务于这一目的。
- 5. 迭代或递归探索：**所有实现都通过某种形式的迭代或递归来深化研究。open_deep_research 通过主管-研究员的多轮交互，gpt-researcher 通过树状递归，agents-deep-research 通过知识盲区驱动的循环，dzhng/deep-research 通过方向驱动的递归。
- 6. 最终报告生成：**所有实现最终都将研究发现综合为结构化的研究报告，通常包含引用来源。

3.2 差异化设计对比

在共性流程的基础上，四个实现在以下关键维度上展现出显著的设计差异。这些差异反映了不同的设计哲学和工程权衡。

3.2.1 编排范式对比

编排范式是四个实现最根本的差异。open_deep_research 采用 LangGraph 状态图驱动的层次化多智能体架构，主管智能体负责战略决策，研究员智能体负责执行，形成清晰的两层分工。gpt-researcher 采用“规划-执行”范式，标准模式为线性流程，深度模式为树状递归。agents-deep-research 采用知识盲区驱动的迭代循环，每一轮都显式识别知识不足并针对性补充。dzhng/deep-research 采用最直接的递归树结构，通过方向提取实现自适应探索。

3.2.2 自适应能力对比

自适应能力——即系统根据中间发现动态调整研究策略的能力——是衡量深度研究系统智能程度的关键指标。agents-deep-research 的知识盲区驱动模式具有最强的显式自适应能力，每一轮都重新评估研究方向。dzhng/deep-research通过方向提取实现隐式自适应，但缺乏显式的质量评估。open_deep_research通过 think_tool 支持战略反思，但反思频率和深度取决于主管智能体。gpt-researcher 的自适应能力最弱——标准模式完全静态，深度模式仅在树状结构内有有限的动态性。

3.2.3 并行处理对比

并行处理能力直接影响研究效率。gpt-researcher 通过 async/await 实现高效的并发搜索和摘要，concurrency 参数可配置。agents-deep-research在两个层面实现并行：工具智能体并行执行，以及 DeepResearcher 模式下多章节并行研究。open_deep_research 通过 max_concurrent_research_units控制并行研究员数量。dzhng/deep-research 的并行能力最弱，搜索基本串行执行。

3.2.4 上下文管理对比

上下文管理是深度研究系统的核心挑战之一——如何在有限的 LLM 上下文窗口内处理大量搜索结果。open_deep_research 的 compress_research机制最为成熟，每个研究员独立压缩发现，主管只处理压缩后的摘要，并具备渐进式截断策略。gpt-researcher 在深度模式中跨分支共享上下文。agents-deep-research 通过 Observations Agent 综合信息。dzhng/deep-research的上下文管理最为简单，所有学习要点直接累积，存在上下文溢出风险。

3.2.5 错误处理与鲁棒性对比

错误处理能力决定了系统在面对网络故障、API 限流、低质量结果等情况下的稳定性。open_deep_research 使用 with_retry 包装关键节点，并具备渐进式截断防止上下文溢出。gpt-researcher 在深度模式中跳过失败查询而非终止流程。agents-deep-research 依赖 Agents SDK 的内置错误处理。dzhng/deep-research 的错误处理最为基础，缺乏系统性的恢复策略。

3.3 综合对比矩阵

下表从十二个关键维度对四个实现进行综合对比，直观展示各方案的优势与不足。

维度	open_deep_research	gpt-researcher	agents-deep-research	dzhng/deep-research
编排范式	层次化 多智能体	规划-执行 +树递归	知识盲区 驱动循环	递归树
自适应能力	中 (think_tool反思)	低 (静态 规划)	高 (盲区 驱动)	中 (方向 提取)
并行处理	可配置 并行研究员	async/await 并发	工具+章节 双层并行	基本 串行
上下文管理	压缩+ 渐进截断	跨分支 共享	Observations 综合	直接累积
错误处理	with_retry +截断	跳过失败 查询	SDK内置 处理	基础
人机交互	内置澄清 环节	无	无 (明确 不澄清)	后续问题 澄清
模型分级	可配置 多模型	gpt-4o-mini +gpt-4o	三层模型 分级	单一模型
工具扩展	MCP协议 强扩展	多检索器 多抓取器	自定义 工具	Firecrawl 固定
代码复杂度	高 (LangGraph)	中高	中	低 (<500行)
报告质量	高	中高	高 (长报告 优势)	中
成本效率	中	高 (0.1-0.4 美元)	中	高
适用场景	复杂定制 企业级	快速部署 成本敏感	长篇报告 高质量	原型开发 学习理解

表 3-1 四大开源深度研究实现综合对比矩阵

3.4 关键发现

通过对比分析，我们可以得出以下关键发现：

- **没有单一方案在所有维度上最优：**每个实现都有其独特的优势领域和明显的短板。open_deep_research 在定制性和工具扩展上最强但复杂度最高；gpt-researcher 在成本效率和工程成熟度上领先但自适应能力弱；agents-deep-research 在自适应和长报告质量上最优但依赖特定 SDK；dzhng/deep-research 在简洁性上无可比拟但功能基础。
- **自适应能力是核心差异化因素：**四个实现中，agents-deep-research的显式知识盲区驱动模式代表了最高的自适应智能，这直接影响了研究的针对性和效率。新框架应在此基础上进一步强化。
- **上下文管理是工程关键：**open_deep_research 的压缩+渐进截断策略是最成熟的上下文管理方案，新框架应借鉴这一设计。
- **鲁棒性普遍不足：**除了 open_deep_research 的重试机制和gpt-researcher 的跳过策略外，大多数实现在错误处理上较为薄弱。MiroFlow 论文的鲁棒 workflow 设计提供了改进方向。
- **质量评估机制缺失：**四个实现都没有显式的质量评估环节——系统不知道自己产出的报告质量如何，也无法基于质量反馈进行改进。这是新框架可以突破的重要方向。

第四章 深度研究的一般流程框架

4.1 一般流程的六阶段模型

基于第三章的共性提取和差异化分析，我们可以抽象出深度研究功能的一般流程框架。该框架包含六个阶段，涵盖了从用户输入到最终报告的完整链路。任何深度研究系统——无论是已有的四个实现还是未来的新框架——都可以映射到这一六阶段模型上。

4.1.1 阶段一：意图理解与规划

这是深度研究的起点。系统接收用户的自然语言查询，分析其研究意图、范围和预期产出。该阶段的核心任务包括：查询分析（理解用户真正想知道什么）、范围界定（确定研究的边界和深度）、计划生成（将复杂查询分解为可执行的子任务或子问题）。open_deep_research 的研究简报生成、gpt-researcher 的研究问题规划、agents-deep-research 的 Planner 大纲生成、dzhng/deep-research 的 SERP 查询生成都属于这一阶段。该阶段的质量直接决定了后续研究的方向正确性。

4.1.2 阶段二：信息检索

基于规划阶段生成的子任务，系统通过搜索引擎 API 检索相关信息。该阶段的核心挑战包括：查询构造（将子任务转化为有效的搜索查询）、搜索执行（调用搜索 API 获取结果列表）、结果过滤（初步筛选相关结果）。所有四个实现都使用第三方搜索 API（Tavily、Serper、Firecrawl 等），但查询构造策略各有不同。gpt-researcher 支持多种检索器，open_deep_research 支持 MCP 工具扩展，体现了该阶段的可扩展性需求。

4.1.3 阶段三：内容提取与处理

搜索返回的通常是 URL 列表和简短摘要，系统需要进一步抓取网页全文并提取有用信息。该阶段的核心任务包括：网页抓取（获取 HTML 内容）、内容解析（HTML 转纯文本）、信息提取（从文本中提取与研究目标相关的关键信息）。gpt-researcher 支持多种抓取器（BeautifulSoup、Selenium、Playwright），适应不同网页类型。dzhng/deep-research 使用 Firecrawl 的搜索+抓取一体化 API 简化了这一阶段。

4.1.4 阶段四：信息综合与知识积累

提取的信息需要被综合、摘要并积累到知识库中。该阶段是上下文管理的核心战场——如何在有限的 LLM 上下文窗口内有效组织和利用大量信息。open_deep_research 的 compress_research 机制通过独立压缩每个研究员的发现来管理上下文。gpt-researcher 对每个资源进行独立摘要。agents-deep-research 的 Observations Agent 综合新信息并提供策略思考。dzhng/deep-research 提取学习要点和新方向。该阶段的设计直接影响系统能处理的信息量和最终报告的深度。

4.1.5 阶段五：迭代深化

基于当前积累的知识，系统决定是否需要继续深入研究。这是深度研究区别于传统 RAG 的核心特征——系统不是一次性检索，而是根据已有发现动态调整研究方向。该阶段的决策逻辑因实现而异：agents-deep-research通过知识盲区识别决定下一步方向，dzhng/deep-research通过方向提取驱动递归，gpt-researcher通过树状结构的depth参数控制迭代层数，open_deep_research通过主管智能体的战略决策控制研究进程。该阶段的自适应程度是衡量系统智能水平的关键指标。

4.1.6 阶段六：报告生成

当研究达到终止条件（知识盲区已覆盖、迭代次数用尽、时间限制等），系统将所有研究发现综合为最终的研究报告。该阶段的核心任务包括：信息组织（按逻辑结构组织研究发现）、报告撰写（生成连贯的叙述性文本）、引用标注（标注信息来源，便于验证）。agents-deep-research的DeepResearcher 模式还有额外的校对环节，确保长报告各章节的一致性。报告质量是用户感知系统价值的直接指标。

4.2 一般流程的可视化

下图展示了深度研究一般流程的六阶段模型及其迭代关系。需要注意的是，阶段二到阶段五构成一个迭代循环——系统在"检索-提取-综合-深化"的循环中不断积累知识，直到满足终止条件后才进入报告生成阶段。



图 4-1 深度研究一般流程的六阶段模型

4.3 关键设计决策

在六阶段模型的基础上，深度研究系统的设计者需要在以下关键决策点上做出选择，这些选择定义了系统的特性和能力边界。

4.3.1 规划策略：静态 vs 动态

静态规划在研究开始时一次性生成所有子任务，执行过程中不修改计划（gpt-researcher 标准模式）。动态规划在执行过程中根据发现不断调整计划（agents-deep-research 的知识盲区驱动）。动态规划具有更强的自适应能力，但实现复杂度更高，且可能引入规划不稳定的问题。

4.3.2 迭代控制：参数驱动 vs 质量驱动

参数驱动通过固定参数（breadth、depth、max_iterations）控制迭代规模（gpt-researcher、dzhng/deep-research）。质量驱动通过评估当前知识的充分性来决定是否继续迭代（agents-deep-research 的知识盲区覆盖判断）。质量驱动更智能但需要可靠的质量评估机制。

4.3.3 上下文管理：累积 vs 压缩 vs 分层

累积策略直接将所有发现放入上下文（dzhng/deep-research），简单但易溢出。压缩策略对发现进行摘要后再放入上下文（open_deep_research 的 compress_research），有效管理上下文但可能丢失细节。分层策略维护不同层级的记忆（工作记忆、情节记忆、语义记忆），在上下文管理和信息保留之间取得平衡，但实现复杂度最高。

4.3.4 并行策略：串行 vs 并行 vs 混合

串行策略逐个执行搜索和处理（dzhng/deep-research），简单但效率低。并行策略同时执行多个搜索和处理（gpt-researcher 的 async/await），效率高但需要并发控制。混合策略在不同层级采用不同策略——如主管串行决策、研究员并行执行（open_deep_research），平衡了效率和可控性。

第五章 MiroFlow 论文的关键启示

5.1 MiroFlow 概述

MiroFlow (arXiv:2602.22808) 是一个高性能的开源智能体框架，专注于深度研究任务。该框架在 GAIA、BrowseComp、HLE、xBench-DS、FutureX 等多个权威基准测试上达到了 SOTA (State-of-the-Art) 性能，超越了此前的开源方案和部分商业产品。MiroFlow 的核心技术贡献在于其三层次化架构和三项关键技术创新，为新框架的设计提供了重要启示。

5.2 三层次化架构

MiroFlow 采用控制层、智能体层和基础层的三层架构。控制层负责整体工作流的编排和调度，包括任务分解、智能体分配和结果聚合。智能体层包含执行具体任务的智能体节点，每个节点是一个可独立调用的智能体。基础层提供底层能力支持，包括 LLM 调用、工具接口和记忆管理。这种分层设计实现了关注点分离，使得各层可以独立演化和优化。

5.3 三项关键技术创新

5.3.1 智能体图 (Agent Graph)

MiroFlow 使用有向图结构来组织智能体协作。图中的节点代表智能体，边代表数据流向和控制依赖。这种图结构比线性的流水线或固定的状态机更加灵活——可以根据任务需求动态调整图的拓扑结构，支持分支、合并、循环等复杂模式。智能体图使得复杂的工作流可以被清晰地表达和管理，同时支持并行执行和条件路由。

5.3.2 鲁棒 workflow (Robust Workflow)

MiroFlow 的鲁棒 workflow 包含三项关键机制：消息规范化、重试机制和故障隔离。消息规范化对智能体之间的通信消息进行标准化处理，减少了 LLM 输出的随机波动对 workflow 稳定性的影响。重试机制对失败的智能体调用进行自动重试，提高了执行的可靠性。故障隔离确保单个智能体的失败不会导致整个 workflow 崩溃，失败会被隔离在局部范围内处理。这三项机制共同保证了系统在面对 LLM 不确定性、网络故障和异常输入时的稳定性。

5.3.3 重推理模式 (Heavy-Reasoning Mode)

对于复杂问题，MiroFlow 采用集成+验证的重推理模式。集成阶段让多个智能体独立解决同一问题，生成多个候选答案。验证阶段通过专门的验证智能体对候选答案进行交叉检验，选择最一致、最可靠的答案。这种模式类似于“多模型投票”但更加精细——验证不仅比较最终答案，还比较推理过程和证据支持。重推理模式显著提高了复杂问题的一致性和可靠性，是 MiroFlow 在基准测试上取得 SOTA 性能的关键因素之一。

5.4 对新框架设计的启示

MiroFlow 的设计理念对新框架 AURA 的设计提供了以下关键启示：

- **图结构编排优于线性流水线：**AURA

应采用有向图结构组织智能体协作，支持灵活的拓扑结构和动态调整。

- **鲁棒性是生产级系统的必备特性：**消息规范化、重试机制和故障隔离应作为 AURA 的基础设施层面能力，而非可选功能。

- **集成+验证模式提升复杂问题质量：**AURA

应在关键决策点引入多智能体集成和交叉验证机制，特别是对于事实性声明和关键结论。

- **三层分离促进可维护性：**AURA

应明确区分控制层、智能体层和基础层，使各层可以独立演化和测试。

- **消息规范化降低随机性：**AURA 应对智能体间通信进行结构化规范，减少 LLM 输出波动对系统稳定性的影响。

核心启示：MiroFlow 的成功证明，深度研究系统的性能提升不仅依赖于更强的 LLM，更依赖于更优的工程架构。鲁棒性、集成验证和灵活编排是构建高质量深度研究系统的三大支柱。AURA 框架应将这三大支柱作为核心设计原则。

第六章 AURA 新框架设计与实现计划

6.1 设计理念与目标

AURA (Adaptive Unified Research Architecture, 自适应统一研究架构) 的设计目标是构建一个在自适应能力、结果质量、执行鲁棒性和成本效率四个维度上全面超越现有方案的开源深度研究框架。AURA 的设计融合了四个开源实现的优秀实践和 MiroFlow 论文的关键技术贡献, 同时引入多项创新设计以突破现有方案的局限。

6.1.1 四大设计目标

- 自适应 (Adaptive)**: 系统应根据查询复杂度和中间发现动态调整研究策略, 而非依赖固定参数。借鉴 agents-deep-research 的知识盲区驱动模式, 并进一步引入质量评分机制实现"质量驱动"的迭代控制。
- 统一 (Unified)**: 提供统一的接口和架构, 支持从快速研究到深度研究、从短报告到长报告的多场景, 无需切换不同框架。借鉴 gpt-researcher 的多模式设计和 agents-deep-research 的双模式设计。
- 鲁棒 (Robust)**: 内置消息规范化、重试机制和故障隔离, 保证系统在面对 LLM 不确定性、网络故障和异常输入时的稳定性。借鉴 MiroFlow 的鲁棒 workflow 设计。
- 高效 (Efficient)**: 通过模型分级、并行处理和智能缓存优化成本和延迟。借鉴 gpt-researcher 的成本优化策略和 agents-deep-research 的三层模型分级。

6.1.2 核心创新点

AURA 相对于现有方案的五大核心创新点:

- 质量驱动的自适应迭代**: 引入显式的质量评估器, 对当前知识的充分性和可靠性进行评分, 基于评分决定是否继续迭代及迭代方向。
- 三层记忆架构**: 工作记忆 (当前迭代上下文)、情节记忆 (历史迭代发现)、语义记忆 (跨研究知识库), 实现高效的上下文管理。
- 引用图与交叉验证**: 构建"声明-来源"的引用图, 对关键声明进行多源交叉验证, 借鉴 MiroFlow 的重推理模式。
- 渐进式报告生成**: 报告在研究过程中渐进式构建, 而非在最后一次性生成, 提高效率并支持中途检查点。
- 成本感知路由**: 根据任务复杂度自动路由到合适的模型层级, 并维护成本预算, 在质量与成本间动态平衡。

6.2 架构总览

AURA 采用三层架构, 借鉴 MiroFlow 的分层设计但进行了扩展和优化。

6.2.1 三层架构

AURA 的架构自上而下分为编排层、智能体层和基础层。

层级	职责	核心组件
编排层 (Orchestration)	workflow编排、策略决策、 质量评估、成本控制	Research Orchestrator Quality Evaluator Cost Controller Progress Tracker
智能体层 (Agent)	执行具体研究任务、 智能体协作、信息处理	Planner Agent Searcher Agent Extractor Agent Synthesizer Agent Writer Agent Verifier Agent
基础层 (Foundation)	LLM 调用、工具接口、 记忆管理、鲁棒性保障	Model Router Tool Registry Memory Manager Message Normalizer Retry & Fault Isolation

表 6-1 AURA 三层架构概览

6.2.2 核心 workflow

AURA 的核心 workflow 由编排层驱动，采用"规划-迭代-验证-生成"的四步循环结构。与现有方案相比，AURA 在迭代循环中引入了显式的质量评估和交叉验证环节。



图 6-1 AURA 核心 workflow

6.3 核心模块设计

6.3.1 编排层：Research Orchestrator

Research Orchestrator 是 AURA 的"大脑", 负责整个研究流程的编排和决策。它维护一个有向图结构的研究计划（借鉴 MiroFlow 的智能体图），图中的节点代表子任务，边代表数据依赖。Orchestrator 根据质量评估器的反馈动态调整图的拓扑——添加新节点（深入探索）、标记节点完成（知识充分）或修改节点优先级（调整研究方向）。与 open_deep_research 的主管智能体相比，AURA 的 Orchestrator 不仅做战略决策，还显式管理研究计划的数据结构，使得决策过程可追溯、可调试。

6.3.2 编排层：Quality Evaluator

Quality Evaluator 是 AURA 的核心创新之一。它在每次迭代后对当前知识库进行多维度质量评估，生成质量评分。评估维度包括：覆盖度（研究计划中各子任务的回答充分程度）、可靠性（信息来源的权威性和一致性）、新鲜度（信息的时效性）、深度（是否触及问题的本质而非表面）。质量评分驱动迭代控制——评分低于阈值时继续迭代，高于阈值时进入报告生成。与 agents-deep-research 的知识盲区识别相比，Quality Evaluator 提供了更全面、更量化的质量评估。

6.3.3 智能体层：六类核心智能体

AURA 定义了六类核心智能体，每类智能体有明确的职责边界：

智能体	职责	模型层级	创新点
Planner 规划智能体	分析查询、生成研究计划、构建子任务图	Reasoning (o3-mini级)	生成可执行的图结构计划，而非简单列表
Searcher 搜索智能体	构造查询、执行搜索、初步过滤结果	Fast (gpt-4o-mini级)	查询优化器 + 多检索器 自动选择
Extractor 提取智能体	抓取网页、提取结构化信息	Fast	基于研究目标的针对性提取，非全文抓取
Synthesizer 综合智能体	综合信息、更新知识库、生成摘要	Main (gpt-4o级)	三层记忆管理 + 增量 更新
Verifier 验证智能体	交叉验证关键声明、检测矛盾信息	Reasoning	引用图构建 + 多数 投票验证
Writer 写作智能体	渐进式报告构建、引用标注	Main	渐进式生成 + 风格 一致性保证

表 6-2 AURA 六类核心智能体

6.3.4 基础层：三层记忆架构

AURA 的三层记忆架构是新框架最重要的创新之一，旨在解决现有方案在上下文管理上的不足。

- **工作记忆 (Working Memory)**：存储当前迭代的上下文信息，包括当前子任务、搜索结果、提取的信息和中间摘要。工作记忆容量有限，每次迭代后会被清理和归档。对应 open_deep_research 的 compress_research 机制，但更加结构化。
- **情节记忆 (Episodic Memory)**：存储历史迭代的发现摘要，按迭代顺序组织。情节记忆支持跨迭代的信息检索和引用，使得系统可以回顾之前的研究发现。这是现有方案普遍缺失的层级。
- **语义记忆 (Semantic Memory)**：存储跨研究的结构化知识，包括实体、关系和事实声明。语义记忆使得系统可以积累和复用知识，避免重复研究相同主题。这是面向长期使用的高级特性。

6.3.5 基础层：引用图与交叉验证

AURA 引入引用图 (Citation Graph) 来管理研究发现的可信度。引用图是一个二部图，一侧是声明 (Claim)，另一侧是来源 (Source)，边表示“某来源支持某声明”。当一个声明被多个独立来源支持时，其可信度较高；当来源之间存在矛盾时，系统会标记冲突并触发 Verifier 智能体进行深入验证。这一机制借鉴了 MiroFlow 的重推理模式，但将其从“多智能体投票”扩展为“多来源交叉验证”，更适合深度研究场景。

6.3.6 基础层：鲁棒性保障

AURA 将 MiroFlow 的鲁棒 workflow 设计作为基础层能力，包含三项机制：

- **消息规范化**：所有智能体间通信采用结构化 JSON Schema，对 LLM 输出进行解析和验证，不符合 Schema 的输出会被自动修复或重试。这减少了 LLM 输出随机性对系统稳定性的影响。
- **重试机制**：所有外部调用 (LLM API、搜索 API、网页抓取) 都包装在重试机制中，支持指数退避和最大重试次数配置。
- **故障隔离**：单个智能体或子任务的失败被隔离在局部范围内，不会导致整个 workflow 崩溃。失败的子任务会被标记并跳过，Orchestrator 可以决定是否重新规划替代路径。

6.4 关键创新点详解

6.4.1 质量驱动自适应迭代

AURA 的质量驱动迭代是其相对于现有方案的核心优势。现有方案的迭代控制要么基于固定参数（gpt-researcher 的 breadth/depth、dzhng/deep-research 的递归层数），要么基于简单的知识盲区判断（agents-deep-research）。AURA 引入多维质量评分，使得迭代决策更加精细和智能。

质量评分公式综合考虑四个维度： $Q = w1 * \text{覆盖度} + w2 * \text{可靠性} + w3 * \text{新鲜度} + w4 * \text{深度}$ 。权重 $w1-w4$ 可根据查询类型动态调整——事实性查询侧重可靠性和新鲜度，分析性查询侧重覆盖度和深度。当 Q 低于阈值 $Q_{\min}$ 时，系统继续迭代并基于各维度得分决定迭代方向（如覆盖度低则扩展搜索范围，深度低则针对特定子任务深入探索）。当 Q 高于 $Q_{\min}$ 时，系统进入报告生成阶段。

6.4.2 渐进式报告生成

现有方案都在研究完成后一次性生成报告，这导致两个问题：一是最终生成步骤的上下文压力极大（需要处理所有研究发现），二是用户在研究过程中无法看到任何中间产出。AURA 采用渐进式报告生成——Writer 智能体在每轮迭代后更新报告草稿，将新发现整合到已有结构中。这不仅分散了上下文压力，还支持中途检查点——用户可以在研究过程中查看当前报告草稿，并在必要时提供反馈调整研究方向。

6.4.3 成本感知路由

AURA 的 Model Router 根据任务复杂度自动选择模型层级。简单任务（如查询构造、结果过滤）路由到 Fast 层（gpt-4o-mini 级），标准任务（如信息综合、摘要生成）路由到 Main 层（gpt-4o 级），复杂任务（如规划、验证、质量评估）路由到 Reasoning 层（o3-mini 级）。此外，系统维护成本预算，当累计成本接近预算时自动降级模型层级或提前终止迭代。这一机制借鉴了 agents-deep-research 的三层模型分级，但增加了动态路由和预算控制。

6.5 实现路线图

AURA 的实现分为四个阶段，每个阶段产出可独立验证的里程碑。

6.5.1 阶段一：基础框架（第 1-3 周）

目标：搭建三层架构骨架，实现基础研究流程。

- 实现基础层：Model Router、Tool Registry、Memory Manager 骨架
- 实现消息规范化和重试机制
- 实现 Planner、Searcher、Extractor、Synthesizer、Writer 五个基础智能体
- 实现线性工作流（无迭代）：查询 → 规划 → 搜索 → 提取 → 综合 → 报告

- 支持 Tavily 和 Serper 两种搜索后端
- **里程碑**：能完成简单查询的端到端研究报告

6.5.2 阶段二：迭代与质量（第 4-6 周）

目标：实现质量驱动的迭代循环和三层记忆架构。

- 实现 Quality Evaluator（多维度质量评分）
- 实现迭代循环：搜索 → 提取 → 综合 → 质量评估 → 决策
- 实现工作记忆和情节记忆
- 实现 Research Orchestrator 的动态计划调整
- 实现并行搜索（async/await）
- **里程碑**：能自适应地深入研究复杂查询，质量评分驱动迭代终止

6.5.3 阶段三：验证与鲁棒性（第 7-9 周）

目标：实现引用图、交叉验证和完整的鲁棒性保障。

- 实现 Verifier 智能体和引用图数据结构
- 实现多源交叉验证和冲突检测
- 实现故障隔离和自动恢复
- 实现渐进式报告生成
- 实现成本感知路由和预算控制
- **里程碑**：报告包含引用图，关键声明经过交叉验证，系统具备生产级鲁棒性

6.5.4 阶段四：优化与扩展（第 10-12 周）

目标：性能优化、多模型支持和高级特性。

- 实现语义记忆（跨研究知识库）
- 支持 OpenAI、Anthropic、Gemini、DeepSeek 等多模型后端
- 支持 MCP 工具协议
- 实现人机交互模式（可选澄清环节）
- 实现长报告模式（多章节并行研究 + 校对）
- 性能优化：缓存、连接池、批处理
- **里程碑**：功能完备的开源深度研究框架，支持多种场景和模型后端

6.6 技术栈建议

AURA 建议采用以下技术栈：

组件	技术选择	理由
编程语言	Python 3.11+	生态丰富、LLM SDK 完善、async 支持好
异步框架	asyncio + aiohttp	原生异步支持，高效并行搜索
LLM 调用	LiteLLM	统一多模型接口，支持 100+ 模型
图结构	NetworkX	成熟的图计算库，支持引用图和计划图
记忆存储	SQLite + 向量数据库 (ChromaDB)	轻量级，情节记忆用 SQLite，语义记忆用向量库
搜索后端	Tavily + Serper + DuckDuckGo	多后端冗余，提高可用性
网页抓取	Playwright + BeautifulSoup	动态页面 + 静态解析
配置管理	Pydantic + YAML	类型安全 + 人类可读配置
可观测性	OpenTelemetry + 结构化日志	标准化追踪，便于调试
测试框架	pytest + pytest-asyncio	异步测试支持

表 6-3 AURA 技术栈建议

第七章 预期优势与评估方案

7.1 AURA 相对于现有方案的预期优势

基于第六章的设计方案，AURA 预期在以下维度上超越现有方案。

7.1.1 自适应能力优势

AURA 的质量驱动迭代机制使其自适应能力显著优于现有方案。与 gpt-researcher 的静态参数控制和 dzhng/deep-research 的固定递归层数相比，AURA 能根据知识质量动态决定迭代次数——简单查询可能 1-2 轮即满足质量阈值，复杂查询可能需要 5-10 轮深入探索。与 agents-deep-research 的知识盲区驱动相比，AURA 的四维质量评分（覆盖度、可靠性、新鲜度、深度）提供了更全面的自适应决策依据，不仅知道“还有什么不知道”，还知道“已知信息的质量如何”。

7.1.2 结果质量优势

AURA 的引用图和交叉验证机制使其结果质量显著优于现有方案。现有方案的报告通常只标注信息来源但不验证声明的一致性——如果两个来源对同一事实给出矛盾信息，系统通常不会发现或处理这种矛盾。AURA 的 Verifier 智能体会主动检测矛盾，触发深入验证，并在报告中标注争议点和验证结果。此外，渐进式报告生成使得 Writer 智能体可以在每轮迭代中优化报告结构和叙述连贯性，而非在最后一次性处理所有信息，这有助于生成更连贯、更有深度的报告。

7.1.3 执行鲁棒性优势

AURA 将 MiroFlow 的鲁棒 workflows 设计作为基础层能力，使其执行鲁棒性显著优于现有方案。消息规范化减少了 LLM 输出随机性导致的解析失败；重试机制处理了网络故障和 API 限流；故障隔离确保单个子任务失败不影响整体流程。相比之下，dzhng/deep-research 几乎没有错误处理，gpt-researcher 仅跳过失败查询，agents-deep-research 依赖 SDK 内置处理，open_deep_research 有重试但缺乏故障隔离。AURA 的鲁棒性保障使其更适合生产环境部署。

7.1.4 成本效率优势

AURA 的成本感知路由使其在成本效率上具有优势。通过将简单任务路由到低成本模型、复杂任务路由到高能力模型，AURA 避免了“一刀切”使用高端模型带来的成本浪费。此外，质量驱动的迭代控制避免了过度研究——当质量已满足要求时及时停止，而非机械地执行完所有迭代。语义记忆的跨研究知识复用也能减少重复搜索的成本。预期 AURA 的单次研究成本与 gpt-researcher 相当（0.1-0.4 美元），但在复杂查询上的质量显著更高。

7.2 评估方案

7.2.1 定量评估

AURA 的定量评估将使用以下基准测试和指标：

· **GAIA 基准：**通用 AI 助手能力评估，包含多步骤推理和工具使用任务。MiroFlow 在此基准上达到 SOTA，AURA 的目标是在此基准上达到或超越 MiroFlow 的性能。

· **BrowseComp**

基准：专门评估网页浏览和深度搜索能力，是深度研究系统的核心评估基准。

· **自定义研究质量评估：**构建包含 100

个复杂研究查询的评估集，由人类专家对报告的覆盖度、准确性、深度和引用质量进行 1-5 分评分。

· **成本与延迟指标：**记录每次研究的 API 调用成本和端到端延迟，与 gpt-researcher 和 open_deep_research 进行对比。

· **鲁棒性指标：**在注入故障（API 超时、返回异常、网络中断）的条件下测试系统的恢复能力，统计任务完成率和平均重试次数。

7.2.2 定性评估

除了定量评估，AURA 还将进行定性评估：

· **消融实验：**分别移除质量评估器、引用图、三层记忆等创新组件，评估各组件对整体性能贡献。

· **用户研究：**邀请 20 位用户使用 AURA 和 gpt-researcher 完成相同研究任务，收集用户满意度评分和主观反馈。

· **案例分析：**选取 10 个典型研究查询，详细分析 AURA 的研究过程（迭代轨迹、质量评分变化、引用图演化），与现有方案进行对比。

7.3 预期性能目标

基于设计分析和与现有方案的对比，AURA 的预期性能目标如下：

指标	gpt-researcher	open_deep_research	agents-deep research	AURA（预期）
GAIA 准确率	~50%	~55%	~60%	>65%
BrowseComp 准确率	~30%	~35%	~40%	>45%
报告质量（1-5分）	3.5	3.8	4.0	>4.3
单次研究成本	\$0.1-0.4	\$0.3-0.8	\$0.2-0.6	\$0.15-0.5
端到端延迟	3-5 min	5-10 min	4-8 min	3-7 min
故障恢复率	~85%	~90%	~88%	>95%
引用准确率	~80%	~85%	~82%	>90%

表 7-1 AURA 预期性能目标与现有方案对比

7.4 风险与缓解

AURA 的实现面临以下主要风险，需要提前制定缓解策略：

- **质量评估器的设计难度**：多维度质量评分的权重设定和阈值调优需要大量实验。缓解策略：先实现简单的覆盖度评估，逐步引入其他维度；使用人类标注数据校准评分模型。
- **三层记忆的工程复杂度**：三层记忆的交互和同步可能引入复杂的状态管理问题。缓解策略：先实现工作记忆和情节记忆，语义记忆作为后续扩展；使用成熟的状态管理库。
- **成本控制的平衡**：过于激进的成本控制可能影响质量。缓解策略：设置质量下限，成本控制不得低于质量下限；提供多种预设配置（经济、标准、深度）供用户选择。
- **多模型兼容性**：不同 LLM 的能力和接口差异可能影响一致性。缓解策略：使用 LiteLLM 统一接口；为关键智能体提供模型特定的提示词优化。

7.5 最终结论

深度研究作为大语言模型智能体的核心应用，其框架设计仍有巨大的优化空间。通过对 langchain-ai/open_deep_research、assafelovic/gpt-researcher、qx-labs/agents-deep-research、dzhng/deep-research四个开源实现的系统对比，我们提取了深度研究的六阶段一般流程，并识别出现有方案在自适应能力、质量评估、鲁棒性和成本控制上的改进空间。MiroFlow 论文的鲁棒工作流和重推理模式为新框架设计提供了重要启示。

AURA 框架通过"质量驱动的自适应迭代"、"三层记忆架构"、"引用图与交叉验证"、"渐进式报告生成"和"成本感知路由"五大创新，有望在自适应能力、结果质量、执行鲁棒性和成本效率四个维度上全面超越现有方案。AURA 的价值不仅在于单次研究的质量提升，更在于为开源社区提供一个可扩展、可演进的基础设施，推动深度研究技术的持续进步。我们期待 AURA 能够成为开源深度研究生态的新标杆。